Лабораторийн ажил №5

Сэдэв: Threads, Pthreads, Race condition, Mutex

Гүйцэтгэсэн: Н.Мөнхпүрэв, 23B1NUM0114

1. ОРШИЛ

Энэ лабораторийн ажилд Linux дээрх POSIX thread (Pthreads) ашиглан олон урсгалтай програм бичих аргыг судалсан. Process ба thread-ийн ялгаа, thread үүсгэх pthread_create(), дуусахыг хүлээх pthread_join() функцүүдийг ашигласан.

Хамгийн гол нь нэг shared хувьсагчид хэд хэдэн thread зэрэг хандахад үүсдэг race condition-ийг туршиж харж, дараа нь mutex ашиглан зассан. Эцэст нь банкны дансны бие даасан програм бичсэн.

2. ЗОРИЛГО

Process ба thread-ийг ялгаж ойлгох, Pthreads ашиглан програм бичих, race condition-ийг ажиглаж mutex-ээр шийдэх.

1. Ubuntu дээр gcc-ээр C програмыг -pthread сонголттой хөрвүүлж ажиллуулах.
2. pthread_create() ба pthread_join() ашиглан 3 thread үүсгэх.
3. Синхрончлолгүй shared counter дээр race condition-ийг туршиж ажиглах.
4. pthread_mutex ашиглан critical section-ийг хамгаалж зөв дүн гаргах.
5. Банкны данс (deposit ба withdraw thread) програмыг бие даан бичих.

3. ОНОЛЫН СУДАЛГАА

3.1 Process ба Thread

Process нь ажиллаж буй програм бөгөөд өөрийн виртуал санах ойтой. Thread нь тухайн process доторх гүйцэтгэлийн урсгал юм. Нэг process-ийн thread-үүд код, global хувьсагч, heap-ийг хуваалцдаг, харин thread бүр өөрийн stack болон локал хувьсагчтай. Тиймээс thread хоорондын өгөгдөл солилцоо хурдан боловч shared өгөгдөлд хандахдаа болгоомжтой байх ёстой.

Үзүүлэлт	Process	Thread
Санах ой	Өөрийн виртуал хаягийн орчинтой	Код, heap, global өгөгдлийг хуваалцана
Stack	Process бүр өөрийн stack-тэй	Thread бүр өөрийн stack-тэй
Өгөгдөл солилцоо	IPC механизм хэрэгтэй	Shared memory-д шууд хандана

3.2 Race condition ба Mutex

Race condition гэдэг нь хэд хэдэн thread нэг shared өгөгдөлд зэрэг хандаж, үр дүн нь thread-үүд ажиллах дарааллаас хамаарч буруу болох нөхцөл юм. counter++ нь нэг үйлдэл биш, (1) санах ойгоос унших, (2) 1 нэмэх, (3) буцааж бичих гэсэн 3 алхамтай. Хоёр thread нэг утгыг уншаад тус тусдаа нэмж бичвэл нэг нэмэлт алга болно.

Critical section гэдэг нь shared өгөгдөлд хандах кодын хэсэг. Mutex нь түгжээ бөгөөд critical section-д нэг цагт зөвхөн нэг thread орохыг зөвшөөрнө. Thread lock авсан бол бусад thread unlock хийх хүртэл хүлээнэ.

4. ХЭРЭГЖҮҮЛЭЛТ

Дадлага бүрийг тусдаа эх файлд бичиж, жижиг Makefile ашигласан.

Файл	Үүрэг	Тайлбар
hello_threads.c	Дадлага 1	3 worker thread үүсгэж, join хийнэ.
race_counter.c	Дадлага 2	Mutex-гүй shared counter. Race condition харуулна.
mutex_counter.c	Дадлага 3	Counter-ийг mutex-ээр хамгаална.
bank.c	Бие даалт	Банкны данс: deposit ба withdraw thread.
Makefile	Хөрвүүлэх	make командаар 4 програмыг -pthread-тэй хөрвүүлнэ.

4.1 Дадлага 1: hello_threads

Thread-ийн тоог 3 болгож, ids = {1, 2, 3} өгсөн. Main thread 3 thread үүсгээд pthread_create-ийн кодыг шалгана, дараа нь pthread_join-аар бүгдийг хүлээж, төгсгөлд "Main: buh thread duussan." гэж хэвлэнэ.

4.2 Дадлага 2: race_counter

Thread бүр counter++ үйлдлийг LOOPS удаа mutex-гүй хийнэ. Thread ба LOOPS-ийн тоог -DTHREADS ба -DLOOPS-оор compile хийхдээ өөрчилж болно. Хөрвүүлэхдээ -O0 ашигласан.

4.3 Дадлага 3: mutex_counter

Өмнөх програмтай адил боловч counter++ мөрийг pthread_mutex_lock ба pthread_mutex_unlock хоёрын дунд хийсэн. Функц бүрийн буцаах кодыг check() функцээр шалгана.

4.4 Бие даалт: bank.c

Эхний balance = 1000. Deposit thread 50-г 10 удаа нэмнэ, withdraw thread 30-г 10 удаа хасна. Хоёр thread нэг balance хувьсагч ашигладаг тул mutex-ээр хамгаалсан. Withdraw нь lock авсны дараа balance >= 30 эсэхийг шалгана, хүрэлцэхгүй бол хасахгүй, rejected тоолуурыг нэмнэ. Ингэснээр balance сөрөг болохгүй.

```
static void *withdraw(void *arg) {
    int count = *(int *)arg;
    for (int i = 0; i < count; i++) {
        check(pthread_mutex_lock(&lock), "mutex_lock");
        if (balance >= 30)
            balance -= 30;
        else
            rejected++;
        check(pthread_mutex_unlock(&lock), "mutex_unlock");
    }
    return NULL;
}
```

5. ТУРШИЛТ БА ҮР ДҮН

Орчин: Ubuntu 26.04.1 LTS, gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0, C11, POSIX Threads. Хөрвүүлэх команд:

```
gcc -std=c11 -Wall -Wextra -pthread hello_threads.c -o hello_threads
make
```

5.1 hello_threads (3 thread, 3 удаа)

Ажиллуулалт	Хэвлэх дараалал	Тайлбар
1	Thread 2, Thread 1, Thread 3	Main-ийн мөр эхэнд, "buh thread duussan" төгсгөлд
2	Thread 2, Thread 1, Thread 3	Ижил
3	Thread 2, Thread 1, Thread 3	Ижил

Миний компьютер дээр 3 удаа ижил дараалал гарсан боловч энэ нь баталгаа биш.

Яагаад thread 1 заавал хамгийн түрүүнд хэвлэгдэхгүй вэ?
Thread-үүдийг үйлдлийн системийн scheduler хуваарилдаг. Thread 1 эхэлж үүссэн ч CPU дээр хэзээ ажиллах нь scheduler-ээс хамаарна, тиймээс дараалал өөр байж болно. Join нь зөвхөн Main-ийн сүүлийн мөр бүх thread дууссаны дараа хэвлэгдэхийг баталгаажуулна.

5.2 race_counter

А. 2 thread × 500000:

№	Thread	LOOPS	Хүлээгдсэн	Бодитоор гарсан
1	2	500000	1000000	626836
2	2	500000	1000000	615554
3	2	500000	1000000	501035

Б. 3 thread × 100000 (-DTHREADS=3 -DLOOPS=100000):

№	Thread	LOOPS	Хүлээгдсэн	Бодитоор гарсан
1	3	100000	300000	127313
2	3	100000	300000	136481
3	3	100000	300000	151458

Бүх туршилтад бодит дүн хүлээгдсэнээс бага, мөн ажиллуулах бүрт өөр гарсан. Үүний шалтгаан нь counter++ нь load, add, store гэсэн 3 алхамтай бөгөөд хоёр thread зэрэг уншаад бичихэд нэг

нэмэлт алга болдог. ThreadSanitizer-ээр (-fsanitize=thread) шалгахад counter дээр "data race" гэсэн анхааруулга гарсан (race_counter.c-ийн counter++ мөр: нэг thread уншиж, нөгөө нь бичсэн).

5.3 mutex_counter

Туршилт	Expected	Observed	Дүгнэлт
Mutex run 1	1000000	1000000	Зөв
Mutex run 2	1000000	1000000	Зөв
Mutex run 3	1000000	1000000	Зөв

Яагаад mutex дээр зөв дүн гарч байна вэ?
counter++ нь lock ба unlock-ийн дунд (critical section) байгаа тул нэг цагт зөвхөн нэг thread уншиж, нэмж, бичнэ. Нөгөө thread хүлээдэг тул нэмэлт алдагдахгүй.

Mutex-ийг давталтын гадна байрлуулбал яах вэ?
Дүн зөв хэвээр байна. Гэхдээ нэг thread бүх давталтаа дуустал нөгөө thread хүлээх тул thread-үүд бараг зэрэг ажиллахаа больж, concurrency буурна. Иймд critical section-ийг аль болох богино байлгах нь дээр.

5.4 Банкны програм

Тест	Тохиргоо	Хүлээгдэж буй	Гарсан
A: Үндсэн	balance=1000, +50×10, −30×10	balance=1200, rejected=0	balance=1200, rejected=0
B: Хүрэлцэхгүй	balance=20, зөвхөн withdraw 1 удаа (./bank b)	balance=20, rejected=1	balance=20, rejected=1
C: Давтан	A-г 3 удаа	3 удаа 1200	1200, 1200, 1200

Програмын гаралт (тест A):

```
Final balance: 1200
Rejected withdrawals: 0
```

6. ТАЙЛАНГИЙН АСУУЛТУУД

1. Process ба Thread-ийн 2 гол ялгаа юу вэ?
Process өөрийн тусдаа виртуал санах ойтой, харин нэг process-ийн thread-үүд код, heap, global өгөгдлөө хуваалцана, thread бүр өөрийн stack-тэй. Process хооронд өгөгдөл солилцоход IPC хэрэгтэй, thread-үүд shared memory-г шууд ашиглана.

2. pthread_create() ямар зориулалттай вэ?
Одоо байгаа process дотор шинэ thread үүсгэж, өгсөн функцийг тэр thread дээр ажиллуулна.

3. pthread_join() яагаад хэрэгтэй вэ?

Заасан thread дуустал хүлээнэ. Үгүй бол main эхлээд дуусаж, бусад thread дуусаагүй байхад програм таслагдаж болно. Мөн thread-ийн нөөцийг чөлөөлөхөд хэрэгтэй.

4. Race condition / data race гэж юу вэ?

Хоёр ба түүнээс дээш thread нэг өгөгдөлд зэрэг хандаж, тэдний дор хаяж нэг нь бичих, мөн үр дүн нь ажиллах дарааллаас хамаарч буруу болох нөхцөл.

5. Mutex ямар асуудлыг шийдвэрлэдэг вэ?

Shared өгөгдөлд нэг цагт зөвхөн нэг thread хандахыг баталгаажуулж, race condition (өгөгдлийн алдагдал)-ийг шийднэ.

7. ДҮГНЭЛТ

Энэ лабораторид thread нь нэг process-ийн санах ойг хуваалцдаг тул хурдан хамтран ажиллах боломжтой ч, синхрончлолгүй бол counter++ шиг энгийн үйлдэл хүртэл буруу дүн өгдгийг харлаа (1000000-ийн оронд ойролцоогоор 500000-630000). Mutex ашиглахад дүн 3 удаа бүгд зөв гарсан. Банкны програмд balance-ийг mutex-ээр хамгаалж, шалгалтыг lock дотор хийснээр сөрөг үлдэгдэл үүсэхээс сэргийлсэн. Гэхдээ mutex нь thread-үүдийг түр хүлээлгэдэг тул critical section-ийг богино байлгах хэрэгтэй.

8. ХАВСРАЛТ

ХАВСРАЛТ А. ЭХ КОДУУД

hello_threads.c

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define N 3

static void *worker(void *arg) {
    int id = *(int *)arg;
    printf("Thread %d: Sain baina uu!\n", id);
    return NULL;
}

int main(void) {
    pthread_t threads[N];
    int ids[N] = {1, 2, 3};
    int rc;

    puts("Main: thread-uudiig uusgej baina.");
    for (int i = 0; i < N; i++) {
        rc = pthread_create(&threads[i], NULL, worker, &ids[i]);
        if (rc != 0) {
            fprintf(stderr, "pthread_create: %s\n", strerror(rc));
            for (int j = 0; j < i; j++) pthread_join(threads[j], NULL);
            return EXIT_FAILURE;
        }
    }
    for (int i = 0; i < N; i++) {
        rc = pthread_join(threads[i], NULL);
```

```
        if (rc != 0) {
            fprintf(stderr, "pthread_join: %s\n", strerror(rc));
            return EXIT_FAILURE;
        }
    }
    puts("Main: buh thread duussan.");
    return EXIT_SUCCESS;
}
```

race_counter.c

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/* Тоог -DTHREADS=3 -DLOOPS=100000 гэж compile хийхдээ өөрчилж болно */
#ifndef THREADS
#define THREADS 2
#endif
#ifndef LOOPS
#define LOOPS 500000
#endif

static long counter = 0;

static void *increase(void *arg) {
    (void)arg;
    for (long i = 0; i < LOOPS; i++) {
        counter++; /* zoriud synchronization hiigeegui */
    }
    return NULL;
}

int main(void) {
    pthread_t t[THREADS];
    for (int i = 0; i < THREADS; i++) {
        int rc = pthread_create(&t[i], NULL, increase, NULL);
        if (rc != 0) {
            fprintf(stderr, "pthread_create: %s\n", strerror(rc));
            for (int j = 0; j < i; j++) pthread_join(t[j], NULL);
            return EXIT_FAILURE;
        }
    }
    for (int i = 0; i < THREADS; i++) {
        int rc = pthread_join(t[i], NULL);
        if (rc != 0) {
            fprintf(stderr, "pthread_join: %s\n", strerror(rc));
            return EXIT_FAILURE;
        }
    }
    printf("Expected: %ld, observed: %ld\n", (long)THREADS * LOOPS, counter);
    return EXIT_SUCCESS;
}
```

mutex_counter.c

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
#define LOOPS 500000
static long counter = 0;
static pthread_mutex_t lock;

static void check(int rc, const char *what) {
    if (rc != 0) {
        fprintf(stderr, "%s: %s\n", what, strerror(rc));
        exit(EXIT_FAILURE);
    }
}

static void *increase(void *arg) {
    (void)arg;
    for (long i = 0; i < LOOPS; i++) {
        check(pthread_mutex_lock(&lock), "mutex_lock");
        counter++;
        check(pthread_mutex_unlock(&lock), "mutex_unlock");
    }
    return NULL;
}

int main(void) {
    pthread_t t[2];
    check(pthread_mutex_init(&lock, NULL), "mutex_init");
    for (int i = 0; i < 2; i++)
        check(pthread_create(&t[i], NULL, increase, NULL), "create");
    for (int i = 0; i < 2; i++)
        check(pthread_join(t[i], NULL), "join");
    printf("Expected: %ld, observed: %ld\n", 2L * LOOPS, counter);
    check(pthread_mutex_destroy(&lock), "mutex_destroy");
    return EXIT_SUCCESS;
}
```

bank.c

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

static int balance = 1000;
static int rejected = 0;
static pthread_mutex_t lock;

static void check(int rc, const char *what) {
    if (rc != 0) {
        fprintf(stderr, "%s: %s\n", what, strerror(rc));
        exit(EXIT_FAILURE);
    }
}

/* 50 negjiig count udaa nemne */
static void *deposit(void *arg) {
    int count = *(int *)arg;
    for (int i = 0; i < count; i++) {
        check(pthread_mutex_lock(&lock), "mutex_lock");
        balance += 50;
        check(pthread_mutex_unlock(&lock), "mutex_unlock");
    }
    return NULL;
}

/* 30 negjiig count udaa hasna, uldegdel hureltsehgui bol tatgalzana */
```

```
static void *withdraw(void *arg) {
    int count = *(int *)arg;
    for (int i = 0; i < count; i++) {
        check(pthread_mutex_lock(&lock), "mutex_lock");
        if (balance >= 30)
            balance -= 30;
        else
            rejected++;
        check(pthread_mutex_unlock(&lock), "mutex_unlock");
    }
    return NULL;
}

/* ./bank      -> undsen test (A, C)
   ./bank b    -> test B: balance=20, deposit baihgui, withdraw 1 udaa */
int main(int argc, char *argv[]) {
    pthread_t t1, t2;
    int deposit_count = 10, withdraw_count = 10;

    if (argc > 1 && strcmp(argv[1], "b") == 0) {
        balance = 20;
        deposit_count = 0;
        withdraw_count = 1;
    } else if (argc > 1) {
        fprintf(stderr, "Hereglee: %s [b]\n", argv[0]);
        return EXIT_FAILURE;
    }

    check(pthread_mutex_init(&lock, NULL), "mutex_init");
    check(pthread_create(&t1, NULL, deposit, &deposit_count), "create deposit");
    check(pthread_create(&t2, NULL, withdraw, &withdraw_count), "create withdraw");
    check(pthread_join(t1, NULL), "join deposit");
    check(pthread_join(t2, NULL), "join withdraw");

    printf("Final balance: %d\n", balance);
    printf("Rejected withdrawals: %d\n", rejected);

    check(pthread_mutex_destroy(&lock), "mutex_destroy");
    return EXIT_SUCCESS;
}
```

Makefile

```
CC = gcc
CFLAGS = -std=c11 -Wall -Wextra -pthread
TARGETS = hello_threads race_counter mutex_counter bank

all: $(TARGETS)

hello_threads: hello_threads.c
   $(CC) $(CFLAGS) hello_threads.c -o hello_threads

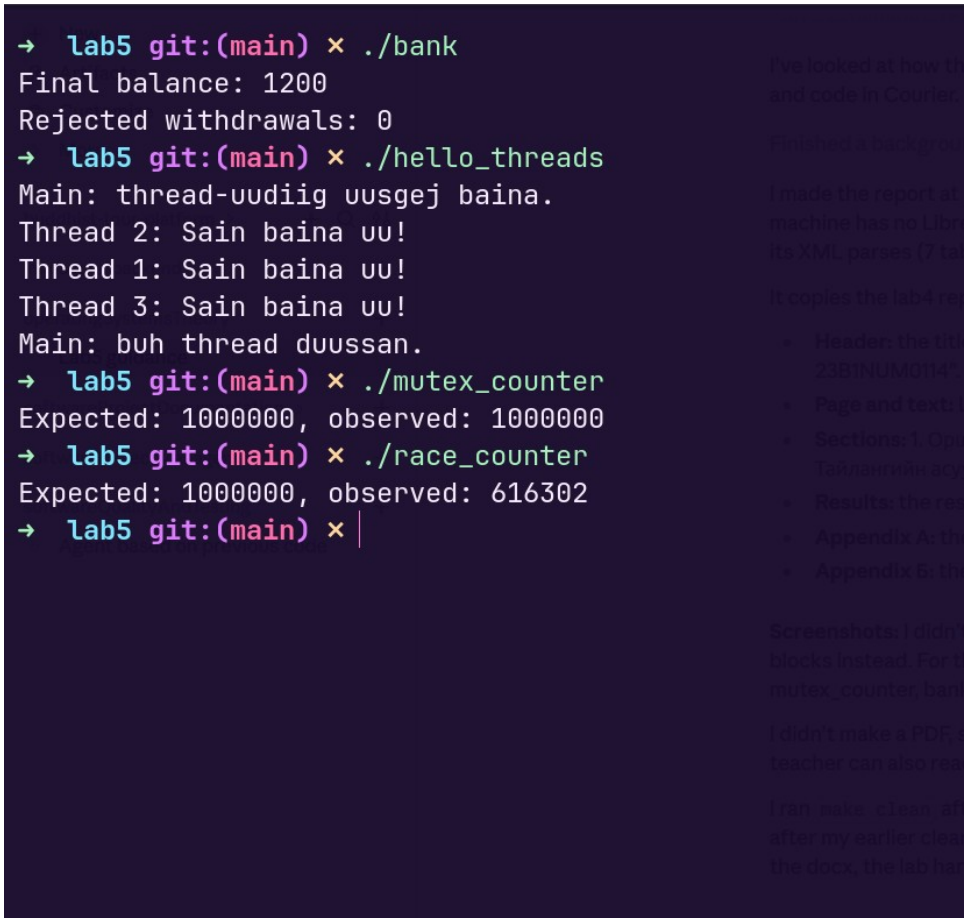
# race_counter-д optimization унтраана (-O0)
race_counter: race_counter.c
   $(CC) $(CFLAGS) -O0 race_counter.c -o race_counter

mutex_counter: mutex_counter.c
   $(CC) $(CFLAGS) mutex_counter.c -o mutex_counter

bank: bank.c
   $(CC) $(CFLAGS) bank.c -o bank
```

```
clean:
	rm -f $(TARGETS) race_tsan
```

ХАВСРАЛТ Б. ПРОГРАМЫН ГАРАЛТ



Зураг 1. bank, hello_threads, mutex_counter, race_counter програмуудын гаралт

hello_threads:

```
Main: thread-uudiig uusgej baina.
Thread 2: Sain baina uu!
Thread 1: Sain baina uu!
Thread 3: Sain baina uu!
Main: buh thread duussan.
```

race_counter (2 thread × 500000):

```
Expected: 1000000, observed: 626836
Expected: 1000000, observed: 615554
Expected: 1000000, observed: 501035
```

mutex_counter:

```
Expected: 1000000, observed: 1000000
Expected: 1000000, observed: 1000000
Expected: 1000000, observed: 1000000
```

bank (./bank b, тест B):

```
Final balance: 20
Rejected withdrawals: 1
```